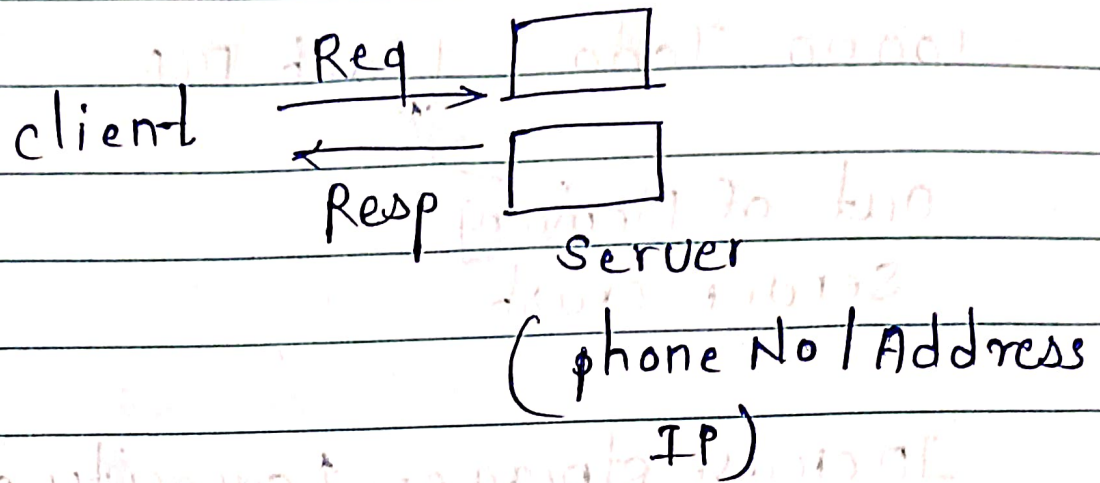


Date: 04/03/26

M T W T F S S

System Design



* DNS Resolutⁿ

find Domain like in IP formate.
save each Domain in specific formate
translate human-Readable domain Name.
into machine readable IP address
to load internet resources.

* For 1 User

Server code

```
const todos = [] // List of Todos
fn createTodo(todoItem) {
  todos.push(todoItem)
}
```

Request



```
const todos = []
fn getAllTodos() {
  return todos
}
```

Response



M T W T F S S

1 Todo = 1 MB

10000 Todo = 1 Lakh MB

out of Memory
Server crash

Increase storage - ↑ capacity of Server
↑ cost per month

* bottleneck

↳ a pt of congestⁿ in system, process workflow that restricts overall through put, efficiency, performance.

* barrigade the Flow (motion)

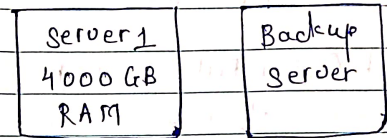
* Vertical scaling: (scaling up)

increase single server's capacity by adding more Resources (CPU, RAM or storage) to handle higher loads.

* Single pt of failure: component such as single server, network link, critical staff member - that upon failing

M T W T F S S

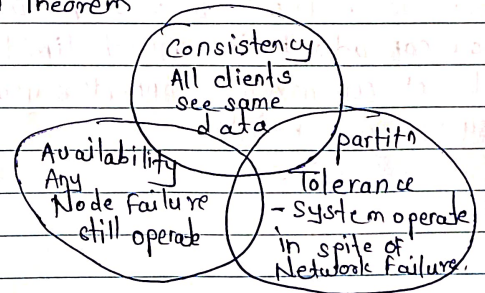
cause the entire system to stop functioning.



If ↗ Data Tradeoff
(Here existing data is not here)
its crash then move to backup server.

* backup Server solves problem for Server Availability but causes Data consistency issue.

* CAP Theorem



M T W T F S S

* Stateful Servers :

Retains client session data (context, login status, shopping cart item) across multiple requests.

* Web socket (stateful servers)

User — Socket
- One to One connectⁿ maintain.

* To scale web socket vertical scaling but disadvantage of vertical scaling
ex, live sports App
more people sign up → more No. of websocket conn.

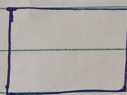
If we just have one websocket server there is a finite amt of resources you can add to it, which limit the No. of concurrent connectⁿs your system can handle.

M T W T F S S

* HTTP Server - Stateless (No memory)

- Real time
- easy for scaling.
- No data.

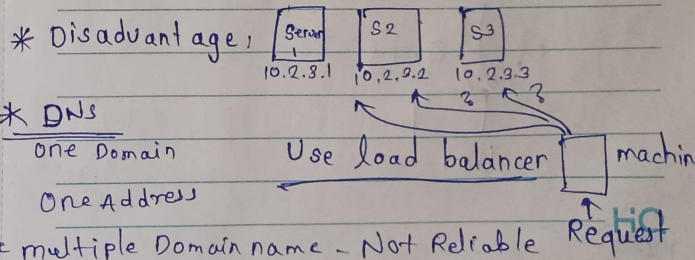
* For consistency we need to use one server.

*  server = can handle 10 users.
If users are increase
Autoscale for Resources

If we have 100 users but 10 Active users so pay for only one server

* Resource planning.

* These scaling is known as Horizontal scaling (Scaling out)

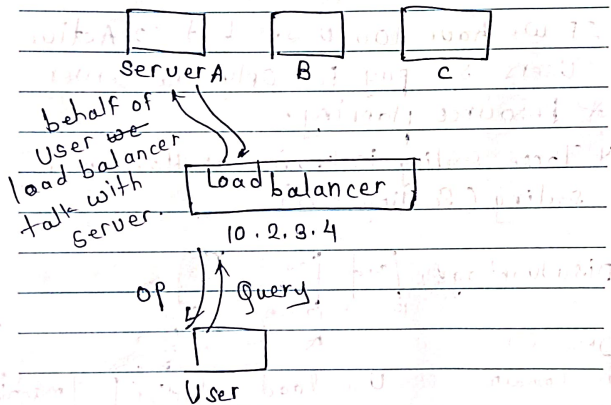


M T W T F S S

* IF we use multiple Domain Name
As Human tendency everyone Use
First one And it crash. so we use
load balancer.

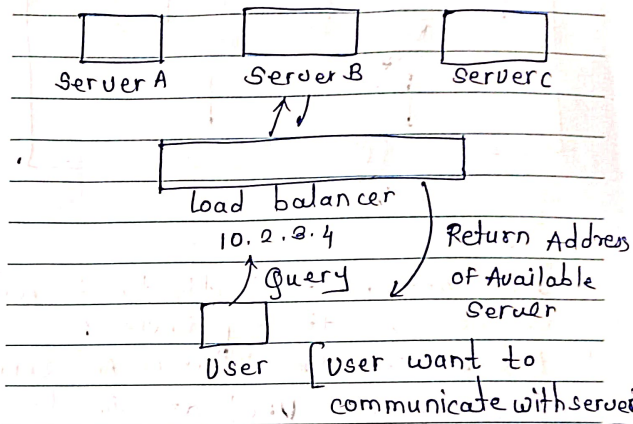
→ a device, software / cloud service
that distributes incoming network
traffic accross multiple servers
to ensure High availability, Fast
response.

Type A)



M T W T F S S

Type B) [Extra Network Hops]



- Not secure

* In Type A we use load balancer as proxy

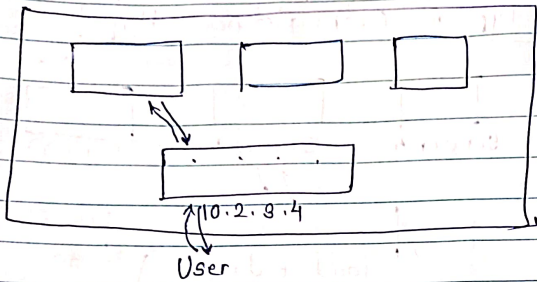
proxy → connect → VPN → Forward Request to google (server)

Reverse proxy

proxy take Request From One User
internally proxy on multiple servers.

Date: 05/03/26

M T W T F S S



* User can't reach directly to servers so we given them private IP And we keep load balancer And servers in same env(VPC) and give load balancer public IP.

* There are less chances to get load balancer down
Inside load balancer - micro machines
↓ That is worker thread.

* AWS ELB (Elastic Load balancer)

Use ML

If our platform uses 10K-1 lakh User so it provision Resources According to it.

* Load balancer:

Limitat^{ns}

① Scaling Delay: Adding new serves to load balancer takes time (1-7mins) before they become fully functional During These period system's capacity to handle traffic temp degrade.

② Sudden Traffic spikes:

Unpredictable surges in traffic ex. live cricket, market events.

③ Out of memory Errors.

insufficient resources.

Solutⁿ: pre-warming. [to handle spike]

- prepare load balancers for High traffic.
- pre-scale its capacity.

step 1 ways

- ① consult cloud providers.
- service providers like AWS offer options to pre warm load balance.
- Understand Requirements For your anticipated traffic.

HQ.

M T W T F S S

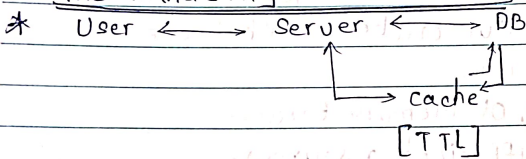
② keep load balancer warm.

CRON Job - Regularly send small amt of traffic to keep load balancer operational.

③ Traffic simulators:

conduct load test to simulate high traffic, verify system performance under stress

* The Thundering Herd Problem



Time to ~~left~~ live

[timestamp that determine ~~terminate~~ cache
How long data packet ~~cha~~ cache exist]

9) IF we set TTL at 6:15 and Any event occur at 6:16 then db get more Request Db load ↑ and Db spike.

M T W T F S S

* solutions:

① Jitter on TTL

set different TTL time for each part of cache

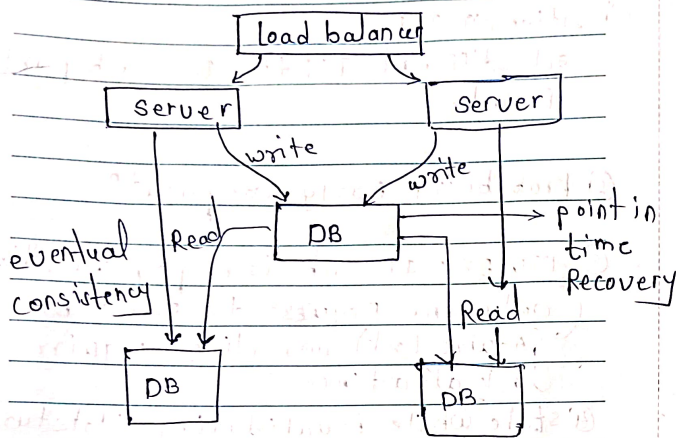
② Probability Early Computatⁿ

③ MUTEX: IF cache is empty we allow only one Request to Reach Db
→ (Acquire Lock) one client acquire lock at a time.

④ stale while revalidating (Hotstar)
Reply New query with old cache
do not empty cache.

⑤ cache warming

M T W T F S S

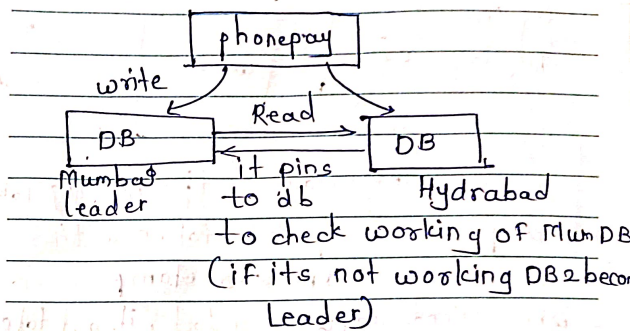


• Point in time Recovery

Roll back to any specific pt in time, down to sec.

M T W T F S S

* The split brain Problem:



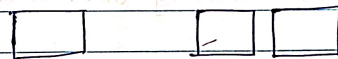
if connection fails between these two DB only one DB records transactⁿ when connectⁿ establish again which transactⁿ we record as final/correct

solutⁿ

① Quorum / Majority Voting

3 nodes / DB

majority nodes votes for write permission.



zookeeper (central nervous system)
(manage, co-ordinate distributed app)

M T W T F S S

② Fencing Token [Monotonically Increasing No. version No / timestamp]

80s-Process

Previlage (h1)

(h2)

write

previlage is the Fencing Token

- each process - separate token with timestamp

- when client acquires a lock, it get token (33)
if client pauses lock expires. Next client
Next token (34)

check : storage Resource (eg. db) keep track of last token.

If old client tries to write with same token (33) it rejects becaz 34 has already been processed
compare higher auth then proceed.

* Single Region critical write
prefer unavailability over inconsistency

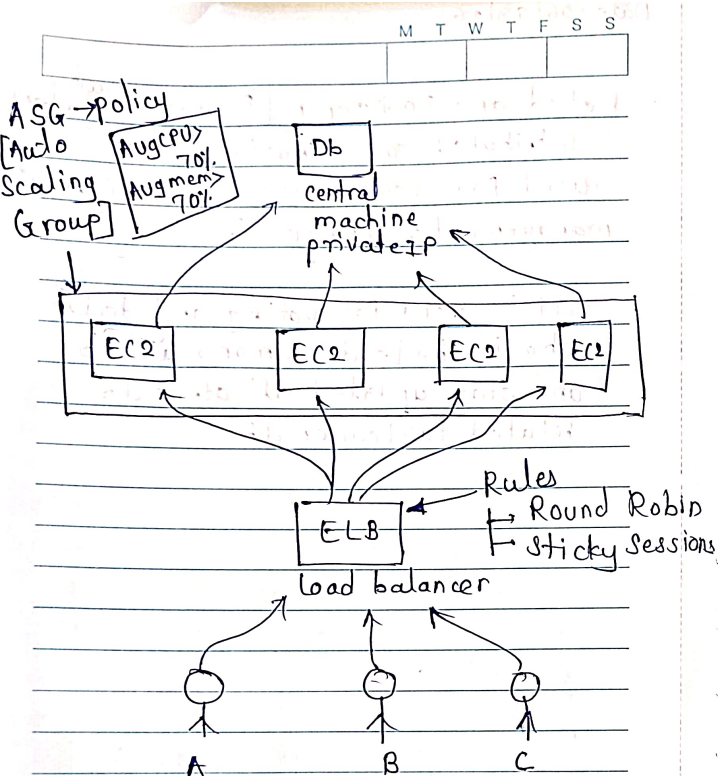
DATE: 06/05/20

M T W T F S S

* etcd and zookeeper [for voting ^{who is leader}]

- distributed co-ordination systems used for leader electⁿ, configuration management & service discovery.

- It is used for voting and decide who is the leader among all db and stay updated all other db related to leader db



MTWTFSS

Ques) If user get response after 10 sec what is the bottleneck here?

- ① Single point of Failure [db is one]
- ② check logs why we are getting late response it will clear us where is the problem accordingly if problem is db so increase Resources

Resources: 2 CPU 4GB Ram → 4 CPU 16GB Ram

* Upgrade db

* problems

① cost ↑

② db down when we scale up system

solⁿ → ① db Replica until master db scale up (Read) multiple. Always write on master db. it take 4-7 secs to update in Read Replicas of db.

* problems

① cost ↑

② storage ↑

③ Data is not Real time.

HQ

M T W T F S S

soln :

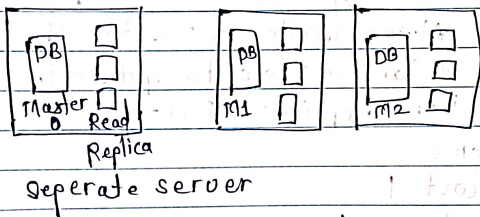
Read Real time data from master db
and Analytics data (not so Imp in
Real time) is Read from Read Replica

ex : insta : $\rightarrow$ post
comments } real time
views } master db
likes } Read Data
From Read
Replica.

* Data inconsistency issue

$\rightarrow$ soln ① Database Sharding

Horizontal partitioning technique
that splits large db into small,
managable pieces (shards)



M T W T F S S

* Hash Fⁿ :

• Function ~~me~~ RetrieveIndex(UserId) {
return UserId % num of servers
}

const index = RetrieveIndex(UserId)
write(db[index])

read from replica (db[index])

If we scale up db

UserId % 3	UserId % 4	db
1 % 3 = 1	1 % 4 = 1	] db
2 % 3 = 2	2 % 4 = 2	
3 % 3 = 0	3 % 4 = 3	
4 % 3 = 1	4 % 4 = 0	
5 % 3 = 2	5 % 4 = 1	
6 % 3 = 0	6 % 4 = 2	
7 % 3 = 1	7 % 4 = 3	

Data Movement

Round Robin data in db index 0
move to db(3)
[overhead, data loss]

M T W T F S S

* Consistent Hashing:

- distributed caching & data partitioning
- Tech that maps data keys & server nodes onto logical circular ring, ensure adding/removing nodes only requires remapping a small fraction of key (K/N).
- It minimize data movement during scaling

* Celebrity Problem: [hotkey or hotspot]

- Single entity (celebrity with millions of followers) causes massive, disproportionate traffic, leading to database bottlenecks & service outage.
- When celebrity posts, millions of followers request that data simultaneously.

Soln

① Caching (Redis)

Date: 10/03/26

M T W T F S S

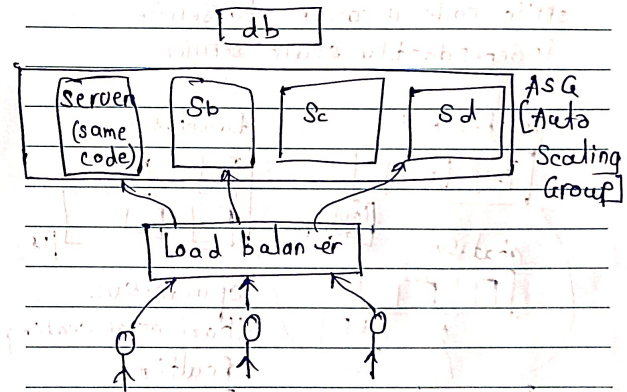
② Read Replicas

③ Application Level Sharding

Instead of one partition for celebrity break their post or followers across multiple db partitions

* Monolith

- whole code is on single Repo.
- soft development model where entire application is built as a single unified & self contained unit



As we grow traffic ↑ so we need to increase servers Redundant code Replicate

M T W T F S S

problem

1) codebase large

- if we increase servers then unnecessary All code is Replicated that means

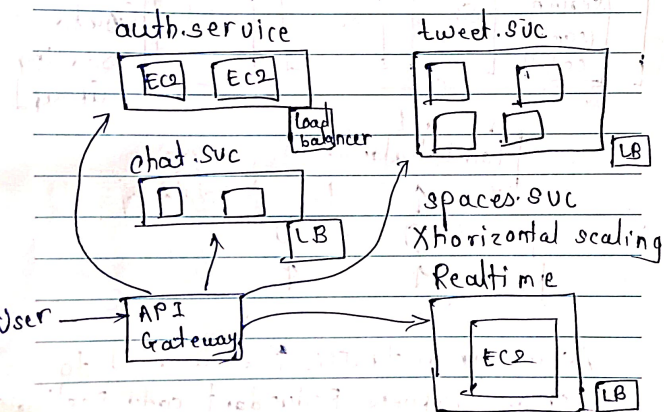
Auth code - Use For 10%.

still we store it for multiple times.

it is not Feature base scaling.

soln: [microservices]

split code according to service.
independently scale service



M T W T F S S

Each load balancer get separate address

* but its not good to have separate domain for each service.

soln

① path base Routing [API Gateway]

base URL domain same

path diff

chai-tweet.com/auth/

/tweet/

Rules For API Gateway

- ① IF Request path is tweet the we point to tweet load balancer

At API Gateway - single pt of failure

- DDOS Attack

(to many users signup)

soln ① Ratelimit.